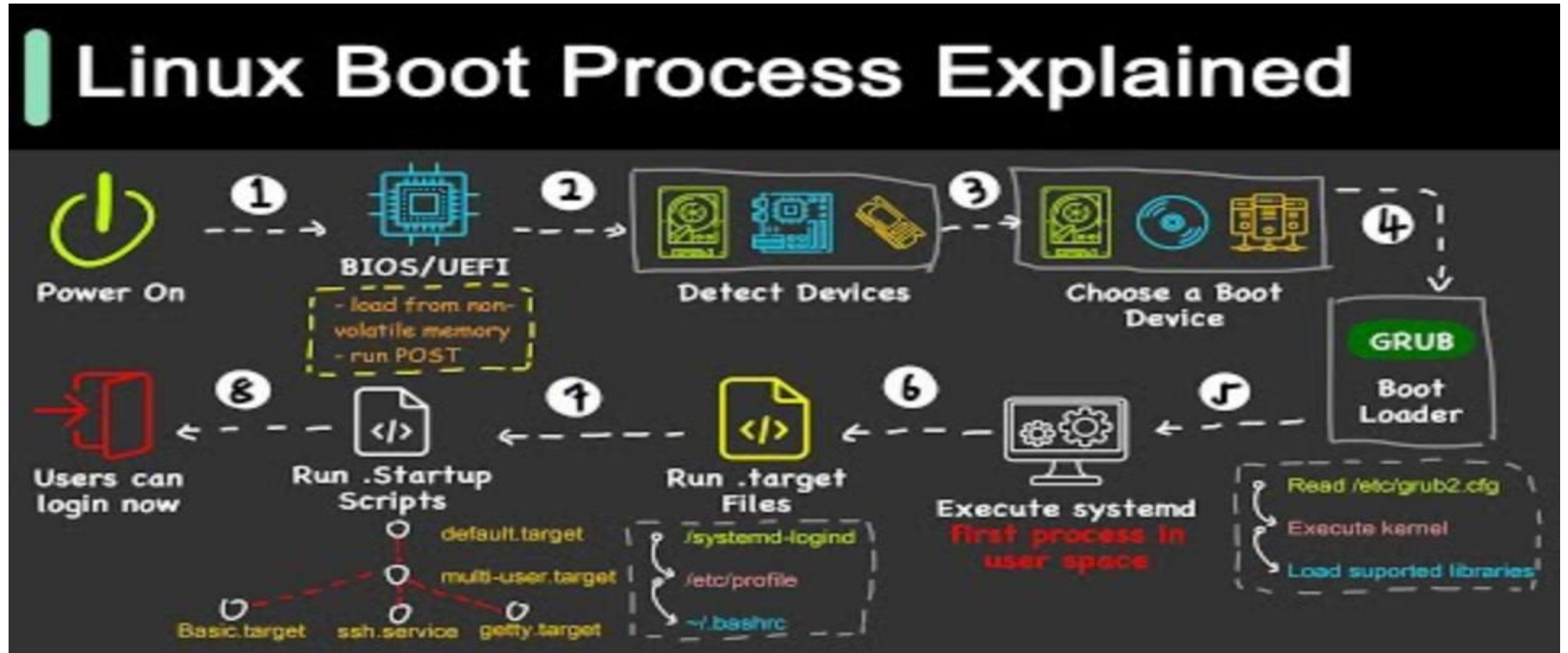


Module IV

LINUX OS FORENSICS

LINUX BOOTING PROCESS



BASICS OF LINUX FILE SYSTEM

- A Linux file system is a structured collection of files on a disk drive or a partition.
- **The general-purpose computer system needs to store data systematically so that we can easily access the files in less time.** It stores the data on hard disks (HDD) or some equivalent storage type.

The **Linux** file system contains the following sections:

- The root directory (/)
- A specific data storage format (EXT3, EXT4, BTRFS, XFS and so on)
- A partition or logical volume having a particular file system.

TYPES OF LINUX FILE SYSTEM

There is a wide range of file systems available in Linux, each of which is designed to meet specific needs. We will learn about some of the most common file systems in Linux below :

- **ext2:**

The ext2 (extended file system 2) is one of the oldest and most widely used Linux file systems. It has been the default file system for most Linux distributions for many years. **The ext2 file system does not have a journal, which means that file recovery can be more difficult after a system crash.** However, it is a stable and reliable file system that is ideal for small to medium-sized partitions.

- **ext3:**

The ext3 (extended file system 3) is an extension of the ext2 file system **that adds journaling support.** Journaling keeps track of changes to the file system, which helps to reduce the risk of data loss after a system crash. The ext3 file system is widely used and is a good choice for most Linux installations. **It is stable, reliable, and has good performance.**

- **ext4:**

The ext4 (extended file system 4) is the successor to the ext3 file system and was designed to address some of the limitations of ext3. **It supports larger file sizes, faster file system checks, and better performance on large disks. It also includes journal checksumming, which improves data integrity. The ext4 file system is the default file system used by many modern Linux distributions.**

- **XFS:**

The XFS(X File System) is a **high-performance file system** designed for large-scale storage systems. It supports file systems up to 16 exabytes in size, making it suitable for use in large data centers. **The XFS file system is known for its scalability, performance, and reliability. It supports advanced features such as journaling, file-level encryption, and online defragmentation.**

- **Btrfs:**

The Btrfs (B-tree file system) is a modern file system designed for use on Linux systems. It supports features such as snapshots, and subvolumes, which allow users to create separate file systems within a single partition. **It also includes built-in support for RAID and compression.**

- **ZFS:**

The **ZFS (Zettabyte File System)** is a powerful and feature-rich file system that was originally developed for Solaris. **It supports advanced features such as snapshots, data compression, deduplication, and built-in RAID. ZFS is known for its reliability and data integrity features.** It is not included in most Linux distributions by default due to licensing issues but can be installed separately.

- **JFS:**

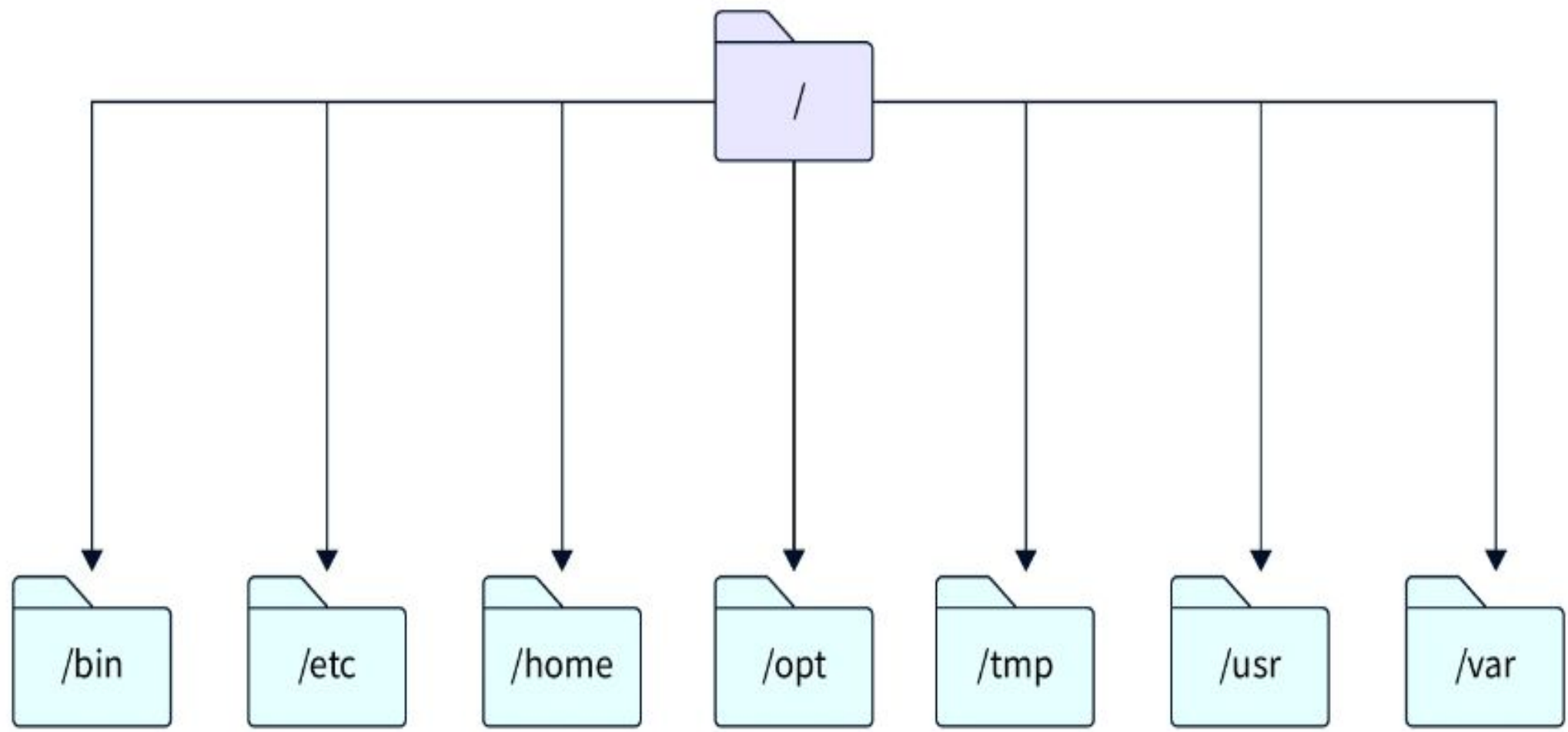
The JFS(Journaled File System) is a high-performance file system that was originally developed by IBM. It includes advanced features such as journaling, file-level compression, and online resizing. **The JFS file system is known for its speed and reliability and is a good choice for high-performance computing systems.**

- **ReiserFS:**

The ReiserFS is a file system that was designed for high-performance computing systems. **It includes advanced features such as journaling, file-level encryption, and support for large files and directories.**

LINUX DIRECTORY STRUCTURE

- **Directories, also known as folders, are an integral part of the Linux file system.**
- **One of the primary functions of directories in Linux is to provide a hierarchical structure for organizing files.**
- **This structure starts at the root directory (/), and branches out into subdirectories as needed.**
- **This makes it simpler to locate and handle files on the system.**



Each directory in Linux file system has a specific function, Some of a few key directories are listed below :

- **/ (root directory):**

The root directory is the top-level directory in the Linux file system. **All other directories and files are contained within the root directory.**

- **/bin:**

The /bin stands for binaries. This directory contains **essential command-line tools and programs that are required for basic system administration tasks.**

- **/boot:**

The /boot directory contains **the boot loader files and kernel images needed to start the system.**

- **/dev:**

The /dev directory contains **device files that represent hardware devices and virtual devices such as terminals, printers, and disks.**

- **/etc:**
The /etc directory **contains system configuration files that are used by various applications and services on the system.**
- **/home:**
The /home directory contains the home directories of users on the system. **Each user has their own subdirectory within /home where they can store their personal files and settings.**
- **/lib:**
The /lib directory **contains shared library files that are needed by various programs on the system.**
- **/media:**
The /media directory **is used to mount removable media such as CDs, DVDs, and USB drives.**
- **/mnt:**
The /mnt directory **is used to mount file systems temporarily, such as network file systems or disk images.**
- **/opt:**
The /opt directory **is used to store additional software packages that are not part of the core system.**

- **/proc:**
The /proc directory is a virtual file system that **provides information about running processes and system resources.**
- **/run:**
The /run directory **contains temporary files that are created by system services and daemons.**
- **/sbin:**
The /sbin directory **contains system binaries and administrative tools that are required for system maintenance**(fdisk, route, reboot, mkfs, init, and fsck)
- **/srv:**
The /srv directory is used to **store data for services provided by the system.**
- **/sys:**
The /sys directory is a virtual file system that **provides information about the system's hardware and devices.**
- **/tmp:**
The /tmp directory **contains temporary files that are created by applications and services running on the system.**
- **/usr:**
The /usr **directory contains user-level programs, libraries, documentation, and shared data files.**
- **/var:**
The /var directory **contains variable data files that change frequently, such as log files and system databases.**

ARTIFACTS IN LINUX SYSTEMS



- On Windows machines, most files of forensic interest like [prefetch files](#), [event logs](#), [file system journal](#), [registry hives](#), etc. can be viewed and processed only using specific tools; either command-line tools or GUI based tools.
- **On Linux machines, most files of forensic interest are plain text files that can be viewed easily in a text editor application.**
- On Windows machines, most forensic operations can be performed via the GUI.
- **On Linux, forensic processes rely on heavy use of the terminal.** For this, the investigator must be well-acquainted with using various command-line tools.

WHAT DATA CAN BE ACQUIRED FROM LINUX COMPUTER?

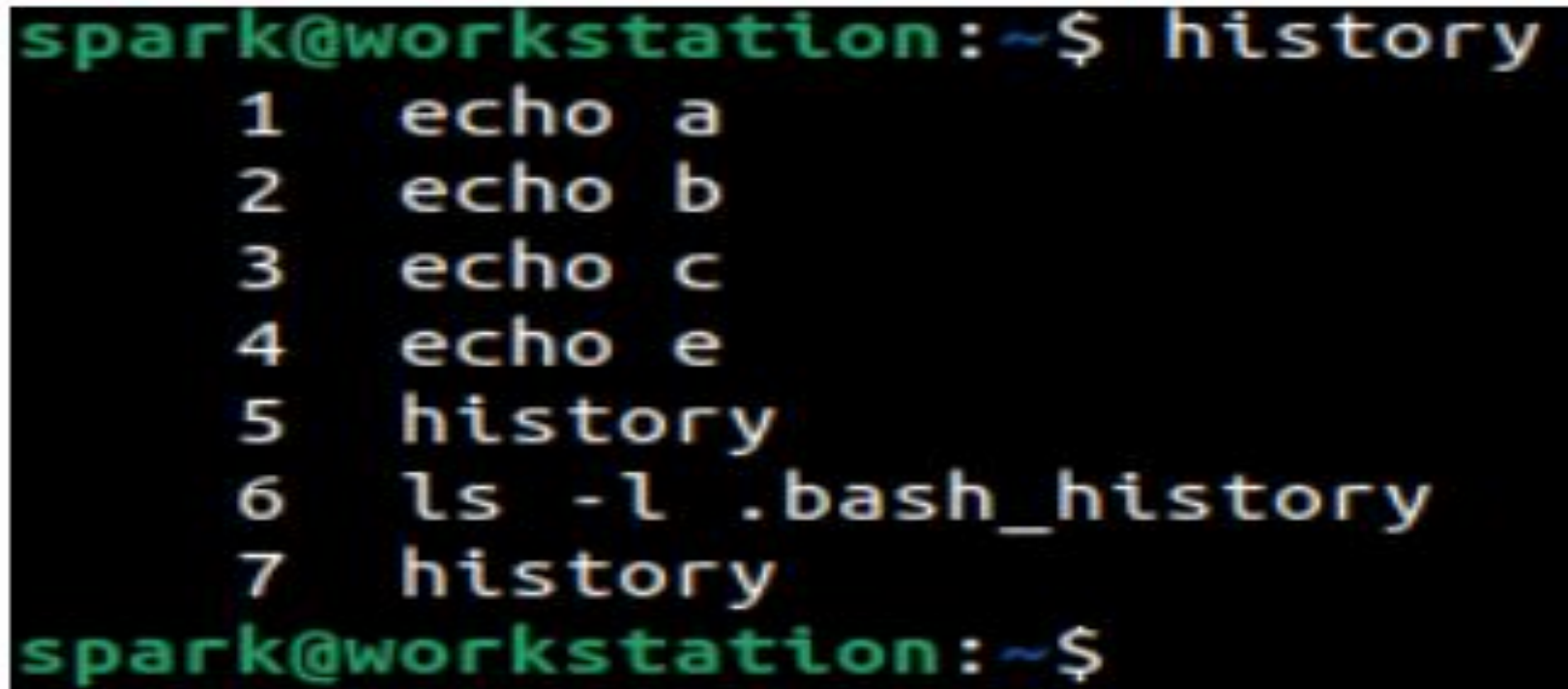
- **System Information:** Due to the various flavors and versions of Linux OS available, when you commence investigation on a system, **start with identifying basic system information like OS version and kernel version.**
- **Memory Dump:** If the system has been found in powered on state, you can begin by acquiring memory from that machine. You will gain access to critical evidence like list of active processes, active network connections, etc.
- **Critical Logs:** **Most of the critical log files are stored within the */var folder*.** Logs relevant to user authentication, installed software packages, kernel logs, system logs, etc. can be found here.
- **Command Line History:** If a system has been configured to store the command line history, then it gives insight into the commands typed into the *Terminal* application recently. Execution of malicious scripts or commands can be identified by processing the command line history.
- **Loaded Kernel Modules:** Some malware load malicious kernel modules to perform its task. Acquiring the list of loaded kernel modules would help track activity on the system.

- **System Configuration:** If the user had installed various services like web server, FTP server, etc. then the configuration used by those services would be stored within **the */etc* folder**. If a service has been suspected of being a part of malicious activity, its stored configuration could provide clues about its capability on the system.
- **User Files:** Every user is assigned a specific */home* directory to store their files. Files would be created, deleted and downloaded. Deleted files can be found within the trash folder. Some critical services also store their files as hidden files, to prevent accidental modification by the user. It is possible to hide files in Linux, simply by adding a dot (.) in front of the file name. All the types of user files serve as critical evidence.
- **External devices:** It is possible to find evidence about external devices attached to the computer like USB devices and external hard disks.
- **Application Data:** Various user applications like web browsers store their own logs. Identifying the applications installed on the system and acquiring the logs stored by those applications would come in handy during an investigation.
- **Networking Data:** Information about the network interfaces on the machine and their configuration will give an idea about the available networking capabilities.

SHELL HISTORY IN LINUX

- We use the ***Terminal*** application on Linux systems regularly to type powerful commands to interact with the system, alter the behaviour of the system, work with files/directories, etc.
- The *Terminal* uses Linux shells in the backend to receive commands from the user, interact with the system and display relevant results to the user.
- **Linux systems are configured by default to store a history of the commands typed by the user, also referred to as *shell history*.**
- Within the **user's *home/* directory**, there is a file called ***.bash_history*** that stores a list of the commands typed on the terminal.

- To view the contents of **.bash_history**, the file can be opened in a text editor or **history command** can be used to view the entries.
- From the following screenshot, we can see the output of history command, displaying the shell commands that had been typed by the user.



```
spark@workstation:~$ history
 1  echo a
 2  echo b
 3  echo c
 4  echo e
 5  history
 6  ls -l .bash_history
 7  history
spark@workstation:~$
```

The screenshot shows a terminal window with a black background and green text. The prompt is 'spark@workstation:~\$'. The user has entered the 'history' command, which displays a list of seven commands: 'echo a', 'echo b', 'echo c', 'echo e', 'history', 'ls -l .bash_history', and 'history'. The list is numbered 1 through 7. The prompt 'spark@workstation:~\$' appears again at the bottom.

- There is another hidden file within the user's ***home/*directory** called ***.bashrc*** that stores configuration information for the shell.

```
spark@workstation:~$ ls -l .bashrc
-rw-r--r-- 1 spark spark 3841 Jun 28 09:14 .bashrc
```

Within *.bashrc*, there is a parameter called HISTSIZE which specifies how many commands can be stored in memory for the current shell session. Parameter HISTFILESIZE specifies how many commands would be stored within the *.bash_history* file.

```
# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000
```

Once 2000 commands have been stored within the shell history file, older entries would be removed as new entries come in.

- Usually, shell history is written into *.bash_history* when the current shell session is terminated.
- This means when you type commands during a session (you may open multiple tabs in the terminal during the session), **the commands are written into the history file only when you close the *Terminal*.**
- If the **system suddenly crashes** when *Terminal* is being used, then shell history for that session is not captured.
- Bash shell history is extremely useful during **DFIR activities to identify which commands had been typed on the system recently.**
- It is possible to find out **if any scripts had been executed, if any services had been started/stopped, if any software programs had been installed, if any commands had been executed, etc.**

LINUX TIMESTAMPS

- One of the most important pieces of data that **can be collected during a digital forensics and incident response investigation is timestamps.**
- Timestamps can be used to help determine **when files were created, accessed, and modified and can also provide insight into the behaviour of an attacker.**
- Linux systems **store timestamps in a variety of different formats and it is important for DFIR practitioners to understand how to interpret these timestamps.**

The four timestamps associated with any file on Linux are:

- **Creation time:** it refers to when the file was created on the system. **It is also referred to as 'birth time'.** It is sometimes abbreviated as btime.
- **Change time:** it refers to time **at which the metadata** associated with the file underwent change. Metadata can refer to timestamps, permissions, ownership, file size, etc. **It is abbreviated as ctime.**
- **Modify time:** it refers to the time **at which the contents of the file** underwent change. Even if a single space or newline character is added to or removed from a file, this timestamp is updated. **It is abbreviated as mtime.**
- **Access time:** it refers to the time at which the file was last accessed. It is abbreviated as atime.

To view all the timestamps associated with the file, `stat` command can be used. In the following screenshot, `stat` is used to view the timestamps for a text file *example.txt*. This file has just been created and is currently empty.

```
spark@workstation:~$ stat example.txt
  File: example.txt
  Size: 6                Blocks: 8          IO Block: 4096   regular file
Device: 807h/2055d      Inode: 3958542    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   spark)   Gid: ( 1000/   spark)
Access: 2022-06-30 10:37:39.873285262 +1000
Modify: 2022-06-30 10:37:37.605106226 +1000
Change: 2022-06-30 10:37:37.605106226 +1000
 Birth: 2022-06-30 10:29:33.584105029 +1000
spark@workstation:~$
```

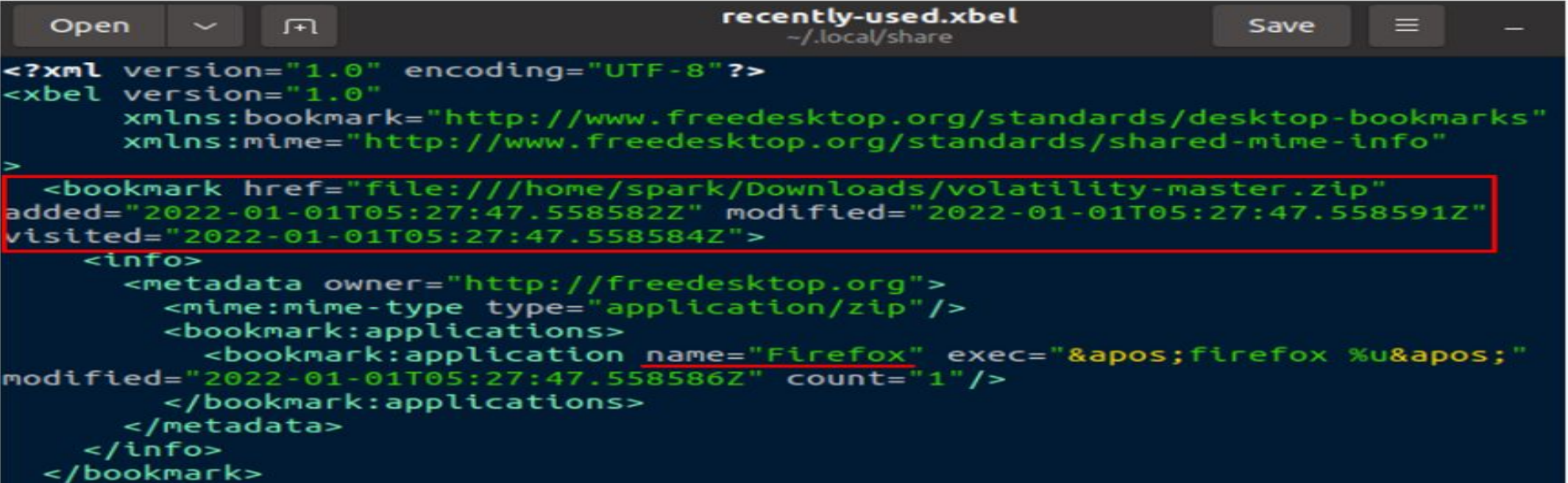
You can view the file name, size, permission string, file owner (spark), group that the file belongs to (spark) and the four timestamps along with time zone information. All the timestamps are the same for the newly created file.

ARTIFACTS IN USER'S HOME/ DIRECTORY

- When a user account is created, a *home/* directory is also created for that user.
- **At the same time, two hidden directories *.local/* and *.cache/* are created within *home/*.**
There are numerous sub-folders within these two directories.

1. Recently Used Files

- Within `/home/[user]/.local/share` there is a file called *recently-used.xbel*, which stores information about which files were recently used on the machine.
- XBEL refers to XML Bookmark Exchange Language, which is a file format that stores every entry as a bookmark.



```
Open  ▾  [icon]  recently-used.xbel  Save  ≡  -  
~/.local/share  
<?xml version="1.0" encoding="UTF-8"?>  
<xbel version="1.0"  
  xmlns:bookmark="http://www.freedesktop.org/standards/desktop-bookmarks"  
  xmlns:mime="http://www.freedesktop.org/standards/shared-mime-info"  
>  
  <bookmark href="file:///home/spark/Downloads/volatility-master.zip"  
added="2022-01-01T05:27:47.558582Z" modified="2022-01-01T05:27:47.558591Z"  
visited="2022-01-01T05:27:47.558584Z">  
    <info>  
      <metadata owner="http://freedesktop.org">  
        <mime:mime-type type="application/zip"/>  
        <bookmark:applications>  
          <bookmark:application name="Firefox" exec="&apos;firefox %u&apos;"  
modified="2022-01-01T05:27:47.558586Z" count="1"/>  
        </bookmark:applications>  
      </metadata>  
    </info>  
  </bookmark>
```


2. Deleted Files

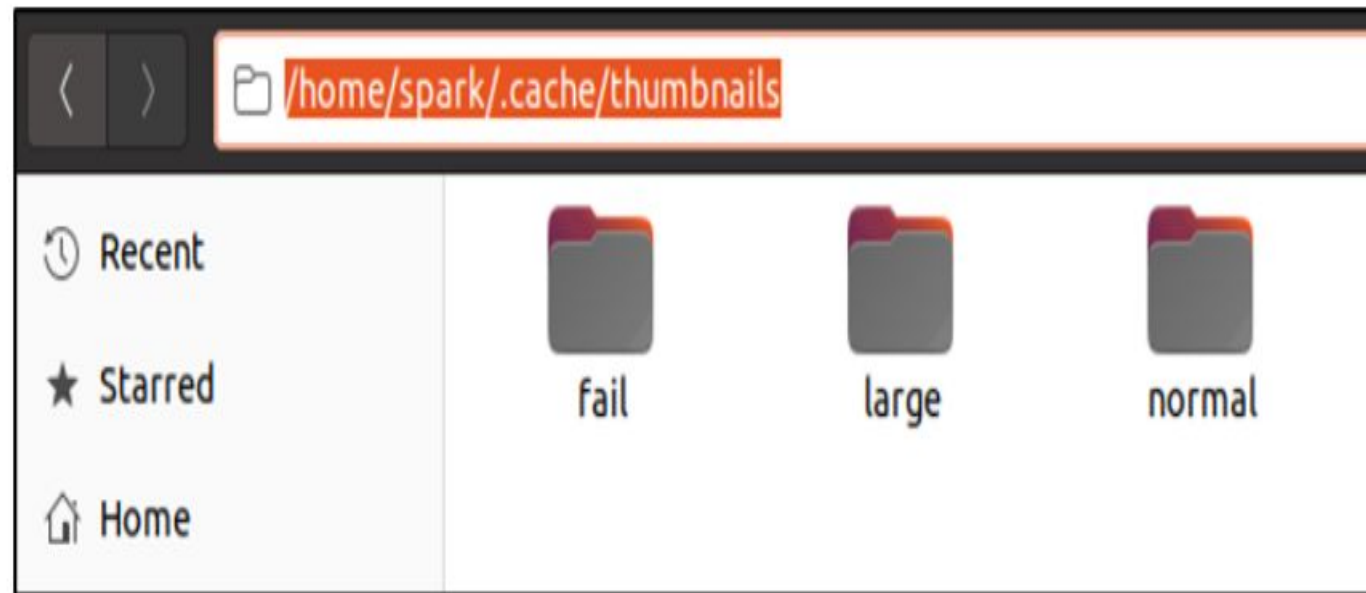
- Whenever a user deletes a file using the delete key or the 'delete' option in the file menu, a copy of the file is placed in the trash bin.
- In addition to that, information about the deletion operation is also stored **within the */home/[user]/.local/share/Trash* directory**, within two sub-directories *files* and *info* as shown in the following screenshot.



- ***/home/[user]/.local/share/Trash/files*** has a copy of all the files and folders deleted by *user*.
- ***/home/[user]/.local/share/Trash/info*** has a file for every deleted file/folder with the extension ***.trashinfo***.
- This file holds the timestamp (relevant to the time zone the device is operating in), of when a file/folder was deleted by the user.

3. Thumbnails

- Thumbnails provide quick previews of image files, PDF files, etc.
- All the thumbnails specific for a user's files are stored in a hidden directory within ***home/ at /home/[user]/.cache/thumbnails***.
- This directory in turn has three sub-directories as shown below, where thumbnails are stored depending on its size.



4. Recently Used GUI Applications

- Within `/home/[user]/.local/share/gnome-shell/`, there is an XML file ***application_state.xml***, that has information about when GUI applications were last used.



ARTIFACTS GENERATED BY MOUNTED DEVICES

- The term *mount* refers to a storage media like hard disk, USB drive or CD-ROM; being placed in a usable state – for a user to create and modify files.
- When a USB drive is attached, it is typically mounted at *D:* or *E:*.
- Likewise, on a Linux computer, apart from the hard disk that stores system information, when external storage media are connected to the computer, **they are typically mounted in the *media/* directory.**
- The hard disk that contains the system files and user files is typically formatted using the Fourth Extended File System, abbreviated as **ext4**.
- Windows assigns names like *C:* to the hard disk having the operating system. **Linux assigns names like *sda*.**

- Every hard disk attached to a Linux computer has a corresponding file for it in *dev/* directory, you can expect to find a file with the name */dev/sda*.
 - Now a hard disk may have many partitions.
 - The partitions would be named as *sda1, sda2, sda3* and so on.
 - If a second hard disk is attached to the same computer, it would be named as *sdb*. You will also find files for each partition within the *dev/* directory.
-
- **Every time external media or file shares are attached or removed, you can find logs regarding the operations within the */var/log/syslog* file.**

LOG SOURCES IN LINUX SYSTEMS

- Every Linux system generates numerous logs by default, as it is being used.

Log Sources

- The main log store in Linux systems is **within the var/ directory**.
- In this section, we will review the **types of logs in /var** and across other locations on the system.
- The following screenshot shows the various logs present within *var/* directory on an Ubuntu system.

```
spark@workstation:/var/log$ ls
alternatives.log      containers            kern.log.1
alternatives.log.1    cups                kern.log.2.gz
alternatives.log.2.gz dist-upgrade         kern.log.3.gz
alternatives.log.3.gz dmesg               kern.log.4.gz
apport.log            dmesg.0             lastlog
apport.log.1          dmesg.1.gz          openvpn
apport.log.2.gz       dmesg.2.gz          pods
apport.log.3.gz       dmesg.3.gz          private
apport.log.4.gz       dmesg.4.gz          speech-dispatcher
apport.log.5.gz       dpkg.log             syslog
apport.log.6.gz       dpkg.log.1           syslog.1
apt                  dpkg.log.2.gz        syslog.2.gz
auth.log              dpkg.log.3.gz        syslog.3.gz
auth.log.1            faillog              syslog.4.gz
auth.log.2.gz          fontconfig.log       teamviewer15
auth.log.3.gz          gdm3                 ubuntu-advantage.log
auth.log.4.gz          gpu-manager.log       ubuntu-advantage-timer.log
bootstrap.log         hp                   ubuntu-advantage-timer.log.1
btmp                  installer            unattended-upgrades
btmp.1                journal              wtmp
clamav                kern.log
spark@workstation:/var/log$
```

All the logs store the timestamps relevant to the time zone the device is operating in. It would be good idea to begin the investigation by *identifying the time zone* settings on the computer.

1. System logs

- On Debian-based distros, logs relevant to global system activity, since system startup can be found at **/var/log/syslog**. It typically stores generic information and non-critical log messages. **On Red-Hat based distros, the equivalent log file is /var/log/messages.**
- Given the amount of data this file collects, it gets very large fairly quickly. To retain the logs, a process called **log rotation** is performed.
- It is where logs present in syslog file are moved to syslog.1 and newly generated logs would be stored in /var/log/syslog.
- **Depending on the configuration of logrotation at /etc/logrotation.conf, the rotation is done weekly or monthly.**
- **Older logs can also be configured to be stored in compressed form.** This is why you see files like syslog.2.gz and syslog.3.gz in the screenshot above.

2. User login logs

- Information about users who have logged into the system, root user actions logins can be found within **/var/log/auth.log** on Debian-based systems and on **/var/log/secure** on Red-Hat based systems.
- This log file undergoes rotation.
- Information about users **currently logged** into the system can be found in the binary file **/var/log/utmp**.
- Information about users who have **logged into the system at any time in the past** can be found in the binary file **/var/log/wtmp**. Timestamps relevant to login and logout can be found.
- **Information about failed login attempts can be found in the binary file /var/log/btmp.**

3. Kernel logs

- Logs relevant to **kernel events** can be found at **/var/log/kern.log**.
- Logs from **kernel ring buffers** and relevant to device drivers can be found at **/var/log/dmesg**. It is typically used to troubleshoot hardware issues.
- Both the files containing kernel logs are quite large and log rotation are typically configured for them.

4. Package Manager logs

- A user may install additional software on their systems using package managers.
- In some cases, installation of one software may install other malicious or suspicious dependencies. On an Ubuntu system, to find information about packages installed using apt, one can look at **/var/log/apt/**.

5. Service logs

- If services like Apache or FTP had been installed on a system, logs for the same can be found **within /var/log**.
- For example: Apache web server's access logs can be found within /var/log/apache2/access.log.
- **Logs generated by the mail server can be found at /var/log/mail.log.**

6. Application Logs

- Multiple applications like web browsers, integrated development environments, etc. are used on a regular basis.
- **User-specific logs for such applications can be found within hidden folders in a user's home/ directory.**
- For examples, say user spark was using **Firefox web browser, you can find browser history, cookies, bookmarks, etc. within files at /home/spark/.mozilla/ directory.**
- Some applications store their logs in **/var/log directory.**



MOSSÉ CYBER SECURITY
INSTITUTE

INVESTIGATE LINUX SYSTEM LOGS



ACQUIRING FORENSIC IMAGE IN LINUX

Acquiring the forensic image of a USB drive using Linux :

- User spark is currently logged into an Ubuntu computer.
- We will be using a command line tool called dd to acquire the forensic image of a USB drive attached to the computer.
- First, let us identify the name assigned to the USB device and its partitions by Linux.
- The command **fdisk** can be used to list all the devices attached to a computer along with information about the partitions on them.
- It was identified that the USB device is assigned the name **sdb**.
- To view information only about the **USB device and its partitions**, **fdisk** was issued again as shown in the following screenshot: **sudo fdisk -l [name-of-device]**

```

spark@workstation:~$ sudo fdisk -l /dev/sdb
Disk /dev/sdb: 1.81 GiB, 1938817024 bytes, 3786752 sectors
Disk model: USB DISK 2.0
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x7187cc1d

Device      Boot    Start        End    Sectors    Size Id Type
/dev/sdb1                2048    2002943    2000896    977M 83 Linux
/dev/sdb2            2002944    3786751    1783808    871M  c W95 FAT32 (LBA)
spark@workstation:~$

```

- Let us take a forensic image of the **entire volume sdb** using **dd**.
- dd was created to copy files from a source to destination.
- The syntax for using the command is as follows: **dd if=[source] of=[destination]**

```
spark@workstation:~$ sudo dd if=/dev/sdb of=full_disk.dd
3786752+0 records in
3786752+0 records out
1938817024 bytes (1.9 GB, 1.8 GiB) copied, 12.1299 s, 160 MB/s
spark@workstation:~$
```

- Upon acquisition of a forensic image, it is a good practice to also store the hash value for it.
- In the following screenshot, we can see that the **SHA512 hash** has been generated for the forensic image of the volume and stored within a text file.

```
spark@workstation:~$ sha512sum full_disk.dd > fulldisk_hash.txt
spark@workstation:~$ cat fulldisk_hash.txt
4281dd745c161df7a621e4d07f6b749e79eb79ffb1217db6d925e6a14cad4967afc2022962
04793239d09ae2c3d161a584b287de3fcf0016f5668c6d5fc5f071  full_disk.dd
spark@workstation:~$
```


PROCESSING A FORENSIC IMAGE IN LINUX

- Once an image has been acquired, we view and process the contents of the forensic image.
- A forensic image of a target computer's hard disk is acquired with the intention of gathering information about recently modified files/directories, confirm the existence of suspicious files or malicious scripts or to look for deleted data.

Every file on a Linux computer has metadata like:

- **a designated type**
- **extension**
- **permissions**
- **size**
- **timestamps**

- We will utilize the **find** command to extract files/directories on the forensic image according to the specified metadata.

Finding files/directories based on Type

- In this section, we will see how find command can be used to identify files based on their type, using the following syntax:

```
find [directory-to-search-within] -type [type]
```

Finding files/directories based on Extension

- The following screenshot shows how find is used to find all the csv files within the mounted forensic image.
- The **-name** option and its argument ***.csv** indicates to search across all the file names in the forensic image and only return results whose file names end with csv extension.

```
spark@workstation:/mnt/part1$ sudo find . -type f -name '*.csv'  
./home/metal/Documents/test-data.csv  
spark@workstation:/mnt/part1$
```

Finding files/directories based on Permissions

- From the following screenshot, we can see how find has been used to print all files, for **which all members of a group can write to**.
- -perm option and its argument is used to specify the permissions to look for.

```
spark@workstation:/mnt/part1$ sudo find . -type f -perm -g=w
./home/metal/Documents/analysis.pdf
./home/metal/Documents/Screenshot (27).png
./home/metal/Documents/file.json
./home/metal/Documents/test-data.csv
./home/element/Downloads/community-master.zip
./home/element/Downloads/jq.zip
spark@workstation:/mnt/part1$
```

Finding files/directories based on Timestamps

- find command can be used to identify files/directories that were modified within the last n number of days using the syntax: **find [directory-to-search] -type [type] -mtime -n**

- The following screenshot shows the list of files modified within the last 60 days, that are present in the forensic image.

```
spark@workstation:/mnt/part1$ sudo find . -type f -mtime -60
./home/metal/Documents/history.log.1.gz
./home/metal/Documents/.data
./home/metal/Documents/test-data.csv
./home/element/Downloads/jq.zip
./var/log/kern.log
./var/log/syslog.4.gz
./var/log/syslog
./var/log/syslog.3.gz
./var/log/dmesg
./var/log/syslog.1
./var/log/dpkg.log
./var/log/auth.log
./var/log/syslog.2.gz
spark@workstation:/mnt/part1$
```

Searching for a specific string

- Assume you want to search for the existence of a specific string across all the files in the forensic image. It is possible with **find** and **grep**.

- The following screenshot shows how find and grep have been used to find all files containing **the word docker** in the forensic image.

```
spark@workstation:/mnt/part1$ sudo find . -type f -exec grep -l "docker" {} \;  
./home/metal/Documents/file.json  
./var/log/kern.log  
./var/log/syslog  
./var/log/syslog.1  
./var/log/auth.log  
spark@workstation:/mnt/part1$
```

Now let's break down this complex command: **find . -type f -exec grep -l "docker" {} \;**

- This search is performed within the **current directory /mnt/part1**.
- **type f** indicates that the search must be performed **across all files in the forensic image**.
- **-exec {} \;** indicates that the **command specified after the exec option must be executed**, which is grep in this case. The string {} is replaced with the name of every file identified by find.
- **grep -l "docker"** indicates to **search for the keyword docker** and print the names of the files in which this keyword is found.

NETWORK ARTIFACTS

- Every Linux computer would be connected to the Internet at some point.
- When a Linux computer is suspected to be involved in an incident, it is critical to collect network artifacts and interpret the recent networking activity that had taken place on the system.

Crucial Network Artifacts

- To gather network artifacts **netstat** command-line tool has been used in the following sections.

Available Network Interfaces

- As seen in the following screenshot, the **-i switch displays the list of available network interfaces on the system.** It provides insight into the possible ways this machine can be connected over a network.

- Here *enp3s0* refers to the ethernet interface, *wlp2s0* refers to the Wi-Fi interface, *lo* refers to the loopback interface, *veth80dc* refers to a virtual network interface (used by hypervisors like VMWare and VirtualBox), *docker0* refers to the virtual network interface created by the Docker installation on user machine.
- The naming convention of the interfaces can vary across Linux distributions.

```
spark@workstation:/var/lib/bluetooth$ netstat -i
Kernel Interface table

```

Iface	MTU	RX-OK	RX-ERR	RX-DRP	RX-OVR	TX-OK	TX-ERR	TX-DRP	TX-OVR	Flg
docker0	1500	2236	0	0	0	2725	0	0	0	BMRU
enp3s0	1500	0	0	0	0	0	0	0	0	BMU
lo	65536	3709	0	0	0	3709	0	0	0	LRU
veth80dc	1500	2236	0	0	0	2785	0	0	0	BMRU
wlp2s0	1500	35197	0	0	0	28279	0	0	0	BMRU

```
spark@workstation:/var/lib/bluetooth$
```


Active Network Connections

- The **-a** switch of **netstat** displays information about all the network connections associated with the system: **tcp** and **udp**.
- In the following screenshot, we can see information about the protocol, its state, local and foreign address where the connection is established or a service is listening.

```
spark@workstation:/var/lib/bluetooth$ netstat -a
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:http-alt        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:50000              0.0.0.0:*               LISTEN
tcp        0      0 localhost:5939          0.0.0.0:*               LISTEN
tcp        0      0 localhost:domain       0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:ssh            0.0.0.0:*               LISTEN
tcp        0      0 localhost:ipp          0.0.0.0:*               LISTEN
tcp        0      0 workstation:40366       239.237.117.34:https    ESTABLISHED
tcp6       0      0 [::]:http-alt         [::]:*                  LISTEN
tcp6       0      0 [::]:50000            [::]:*                  LISTEN
tcp6       0      0 [::]:ssh              [::]:*                  LISTEN
tcp6       0      0 ip6-localhost:ipp     [::]:*                  LISTEN
tcp6       0      0 workstation:47574      maa05s20-in-x0e.1:https ESTABLISHED
udp        0      0 0.0.0.0:45044          0.0.0.0:*
```

- To view hostnames and protocols by their numerical representation, i.e. their IP address (eg, localhost is represented as 127.0.0.1) or port number, then **-n switch** can be used along with **-a**, as shown in the following screenshot.

```
spark@workstation:/var/lib/bluetooth$ netstat -an
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address          State
tcp        0      0 0.0.0.0:8080             0.0.0.0:*                LISTEN
tcp        0      0 0.0.0.0:50000            0.0.0.0:*                LISTEN
tcp        0      0 127.0.0.1:5939           0.0.0.0:*                LISTEN
tcp        0      0 127.0.0.53:53            0.0.0.0:*                LISTEN
tcp        0      0 0.0.0.0:22               0.0.0.0:*                LISTEN
tcp        0      0 127.0.0.1:631            0.0.0.0:*                LISTEN
tcp        0      0 192.168.1.106:40366      34.117.237.239:443       ESTABLISHED
tcp        0      1 192.168.1.106:48526      34.210.39.83:443        SYN_SENT
tcp        0    517 192.168.1.106:48524      34.210.39.83:443        ESTABLISHED
tcp6       0      0 :::8080                  :::*                     LISTEN
tcp6       0      0 :::50000                  :::*                     LISTEN
tcp6       0      0 :::22                     :::*                     LISTEN
tcp6       0      0 :::1:631                  :::*                     LISTEN
```

- To view the associated **process name** and **process ID** for all the listed network connections, then **-p** switch can also be passed to netstat as shown in the following screenshot.


```

spark@workstation:/var/lib/bluetooth$ netstat -anp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:50000           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:5939          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN      -
tcp        0      0 192.168.1.106:43600     108.158.251.74:443     ESTABLISHED 4179/firefox
tcp        0      0 192.168.1.106:40366     34.117.237.239:443     ESTABLISHED 4179/firefox
tcp        0      0 192.168.1.106:48524     34.210.39.83:443       ESTABLISHED 4179/firefox
tcp6       0      0 :::8080                 :::*                   LISTEN      -
tcp6       0      0 :::50000                :::*                   LISTEN      -
tcp6       0      0 :::22                   :::*                   LISTEN      -
tcp6       0      0 :::1:631                :::*                   LISTEN      -

```

Routing Table

- To view the route table of this machine, **then -r switch can be used with netstat;** as shown in the following screenshot.
- When an incident occurs, it is helpful to study this output to infer if the adversary had set up any malicious entries.

```
spark@workstation:/var/lib/bluetooth$ netstat -r
Kernel IP routing table
Destination      Gateway          Genmask         Flags   MSS Window  irtt  Iface
default          _gateway        0.0.0.0         UG      0 0        0     wlp2s0
link-local       0.0.0.0         255.255.0.0     U       0 0        0     wlp2s0
172.17.0.0       0.0.0.0         255.255.0.0     U       0 0        0     docker0
192.168.1.0      0.0.0.0         255.255.255.0   U       0 0        0     wlp2s0
```

ARP Cache

- The ARP (Address Resolution Protocol) cache can be viewed using the **arp -a** command.
- It is helpful to find IP address to MAC address resolution entries here.
- If ARP spoofing is suspected on a network, then investigation of the ARP cache may provide crucial information.

Connected Wi-Fi Networks

- Within the `/etc/NetworkManager/systemconnections` directory, you can find information about all the Wi-Fi networks the system has connected to in the past.
- For each Wi-Fi network a separate file is created, that can be read using cat command.
- Within each file, you can find information about the Wi-Fi SSID, MAC address and security information.

Firewall Rules and Firewall Logs

- For a machine, view the existing firewall rules and the firewall configuration. **The firewall configuration file** may indicate if logging has been enabled for the firewall.
- The TCP wrapper files ***`/etc/hosts.allow`*** and ***`/etc/hosts.deny`*** files also provide a layer of protection for the network.
- It is important to consider these files during an investigation.

Service-specific Logs

- Most Linux systems in a production environment will be running numerous services like Apache, Nginx, SSH, FTP, etc.
- Look within the configuration files of these services for abnormal specifications.
- Assess if the services have been configured to store logs and process the logs.

Bluetooth Artifacts

- Within */var/lib/bluetooth* you can find information about devices paired via Bluetooth to a Linux machine.
- For every paired device, a directory for it is created within */var/lib/bluetooth* with its **MAC address as the directory's name.**
- Creation time of a device's directory indicates when the device was first connected to the system.
- Within the */var/lib/Bluetooth/cache* directory, you can find information about previously paired devices.

ENUMERATING USERS AND GROUPS

- When a Linux computer is suspected to be involved in an incident, one of the important activities to perform is to **enumerate the list of users and groups on the system**.
- It helps to identify any user accounts that stand out as an anomaly.

Viewing details of users on the system

- Every user and group on a Linux system is assigned a user ID (UID) and group ID (GID).
- The ***/etc/login.defs*** file has information about the default UID and GID assigned to the various user accounts on the system.
- The ***/etc/passwd*** file has details about every user account on a Linux system.

Forensic Significance of */etc/passwd* file

- Cyber adversaries may create their own user accounts on a Linux system to function as a backdoor, so that they may gain access to the system whenever they wish.
- It would be best to cross-reference that list against the contents of the */etc/passwd* file to look for outliers.

If an account is considered to be suspicious, then more information must be obtained about it such as:

1. Creation time of */home* directory

- The creation time of a user's */home* directory indicates when the user account was created on the system.
- This information can be obtained using *stat* command

```
spark@workstation:~$ stat /home/spark
  File: /home/spark
  Size: 4096          Blocks: 8          IO Block: 4096   directory
Device: 807h/2055d   Inode: 3932162    Links: 32
Access: (0750/drwxr-x---)  Uid: ( 1000/   spark)   Gid: ( 1000/   spark)
Access: 2022-07-25 11:24:15.627936582 +1000
Modify: 2022-07-19 13:53:50.154627072 +1000
Change: 2022-07-19 13:53:50.154627072 +1000
 Birth: 2022-01-01 14:51:00.257354036 +1100
```


2. Last password change time of a user

- Using **chage command**, as shown below, it is possible to list the password aging information for a particular user.
- It is possible to identify when a user's account password was changed last.

```
spark@workstation:~$ sudo chage -l spark
Last password change                : Jan 01, 2022
Password expires                    : never
Password inactive                   : never
Account expires                     : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
```

If an existing user's account had been compromised and had the password changed recently, the output of this command may provide critical information.

Enumerating Group Membership

- You can find the list of groups on a Linux system by viewing the contents of ***/etc/group*** file as shown below. The output has been truncated here.

```
spark@workstation:~$ cat /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,spark
tty:x:5:
disk:x:6:
```

- The various fields of information that you can see are group name, password, GID and group members.
- Look out for entries that stand out as suspicious.

Password information in **/etc/shadow**

- In the **/etc/passwd** file if x is present in the password field; then a corresponding password value is present in the **/etc/shadow** file.



MOSSÉ CYBER SECURITY
INSTITUTE

HOW TO INVESTIGATE CHROME BROWSER ARTEFACTS ON LINUX

